Universidad Nacional del Altiplano
Facultad de Ingeniería Estadística e Informática
Escuela Profesional de Ingeniería Estadística e Informática

LENGUAJES DE PROGRAMACION I

Estudiante:
Baldarrago Ccoarite Katerine Angela

Lenguajes de Programación

CONCEPTO

Conjunto de reglas sintacticas, semánticas y símbolos formales que permiten a un ser humano escribir instrucciones estructuradas, para que una computadora los interprete, procese y ejecute.

ELEMENTOS ESENCIALES DE UN LENGUAJE DE PROGRAMACIÓN

- Sintaxis: Es el conjunto de reglas gramaticales que dictan como deben escribirse las palabras, simbolos y estructuras para que el código sea válido.

- Semántica: Es el significado o acción lógica que realiza cada instrucción escrita.

- Vocabulario (Palabras reservadas): Palabras clave con significados especiales para el lenguaje que no pueden usarse para otros fines.

  Ej: print, cout, while, etc.

PROGRAMACIÓN

Es el proceso de diseñar escribir probar y mantener código fuente para dar instrucciones a una computadora permitiendole ejecutar tareas específicas o resolver problemas de forma automatizada.

ALGORITMO

Secuencia finita, ordenada y lógica de pasos o instrucciones precisas que deben seguirse para realizar una tarea o resolver un problema concreto.

LENGUAJE DE ALTO NIVEL

Es un lenguaje diseñado para ser leído y escrito fácilmente por los seres humanos. Utiliza palabras en inglés y estructuras lógicas cercanos al lenguaje nativo, independizando al programador de la arquitectura interna de hardware.

- Ej: C++, Pnyton, Javascript, C#.

LENGUAJE DE BAJO NIVEL

Es un lenguaje que opera con mínima o nula abstracción del hardware diseñado para comunicarse e interactuar directamente con la memoria y el procesador de la computadora

- Lenguaje máquina (binario) y lenguaje Assembly (términos mnemónicas).

Entornos de desarrollo Integrados (IDE:)

CONCEPTO

Es una aplicación de software que reúne todas las herramientas necesarias para que un desarrollador escriba y pruebe código en un solo lugar. componentes principales de un IDE:

- Editor de código fuente : Incluye resaltado de sintaxis, autocompletado y detección de errores en tiempo real.
- Compilador \ Interpretador: Integrado permite ejecutar el programa con un solo clic.
- Depurador (Debugger): Herramienta para inspeccionar la memoria, avanzar línea por línea y rastrear errores.
- Gestor de proyectos: Organiza múltiples archivos, bibliotecas y configuraciones del programa.

Variable

Una variable es un espacio reservado en la memoria RAM que se utiliza para almacenar un valor que puede cambiar a lo largo de la ejecución de un programa, tiene un nombre (identificador) y un tipo de dato asociado.

Ejemplo:

Convertir C° a F°

Constante

Una constante es un valor almacenado en memoria que no puede cambiar ni ser modificado una vez que ha sido definido durante la ejecución del programa, se utilizan para evitar errores y representar valores fijos del dominio.

Ejemplo: PI = 3.14151692

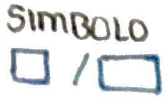
Tipos de datos

Categoría	Tipo de dato	Tamaño habitual	Rango
Enteros	int	4 bytes	-2,147,483,648 a 2,147,483,647
	short	2 bytes	-32.768 a 32.767
	long	4 o 2 bytes	-2,147,483,648
	long long	8 bytes	$-9,22 \times 10^{18}$ a 9.22×10^{18}
Punto flotante	float	4 bytes	$\pm 3.4 \times 10^{38}$
	double	8 bytes	$\pm 1.7 \times 10^{308}$
Caracteres	char	1 bytes	-128 a 127
	wchar	2 a 4 bytes	0 a 65.535
Lógicos	bool	1 byte	0 o 1 (True o false)

DFD

Es una representación gráfica que muestra como fluye la información a través de un sistema o proceso.

Elementos principales:

SIMBOLO	NOMBRE	FUNCIÓN
	Entidad ext.	Representa la fuente de origen de los datos.
	Proceso	Operación o función que transforma los datos.
	Almacen de datos	Lugar donde se guardan los datos temporal o permanentemente.
flecha	Flujo de datos	Muestra la dirección en la que viaja la información entre los elementos.

Pseint

Es una herramienta educativa de software libre diseñada para ayudar a estudiantes a dar sus primeros pasos en la programación y el desarrollo de la lógica algorítmica.

Utiliza pseudocódigo en español, lo que permite centrarse, en los conceptos fundamentales de la programación (variables, condicionales, bucles y funciones) sin la complicación de la sintaxis estricta de un lenguaje real, (como C++, Java, o Python)

Actividad: Conversar de tiempo (segundos a minutos)

↳ Lee un valor en segundos e imprime a cuantos mitutos y horas equivale

Cálculo del IGV de un producto

↳ Solicita el precio base de un producto y calcula el monto del IGV (18%) junto con el precio total final.

Área y perímetro de un rectangulo

↳ Pide la longitud de la base y la altura de un rectangulo para imprimir su area y perimetro

Operadores

Operadores Aritméticos

+ , − , * , / , % , ++ , −−

Operadores Relacionales

== , != , > , < , >= , <=

Operadores lógicos

&& , || , ! (en Pscint : AND , O , NOT)

Estructuras condicionales

If else:

Las estructuras if-else son sentencias de control condicionales que permiten tomar decisiones en el código ejecutado bloques de instrucción únicamente cuando se cumple una condición.

Estructura if (condicional simple):

Evalúa una condición. Si la condición resulta verdadera (true), se ejecuta el código dentro del bloque. Si es falsa (false), el programa omite ese bloque y continúa con la siguiente instrucción.

Estructura if-else (condicional doble)

Permite definir un camino alternativo. Si la condición resulta verdadera (true), se ejecuta el primer bloque. Si es falso (false), se ejecuta obligatoriamente, el bloque dentro del else.

Ej: Par o impar : if (numero %2 == 0)
else elseif impar.

★ Clasificar un número como positivo, negativo o nulo.

Switch - case

En C++, switch/case es una estructura de control de selección múltiple que evalúa el valor de una variable y ejecuta un bloque de código según la coincidencia con una lista de casos enteros o caracteres.

Ejemplo:

```
switch (variable) {
  case var1:
    // código
    break;
  case var2:
    // código
  default:
    // código
    break;
}
```

Ejercicios

Crear un código C++ para la clasificación de glucosa en sangre para la detección de hipoglucemia diabetes o prediabetes.

Completar hasta del 1 al 10 000 en código numeros y letras

Estructuras Repetitivas

También llamado bucle ciclo o iteración. Es un bloque de código que permite ejecutar una o varias instrucciones de forma recurrente mientras se cumpla una determinada condición lógica.

Su propósito principal es automatizar tareas repetitivas, evitar la duplicidad de código y procesar colecciones de datos (como listas o arreglos) de manera eficiente.

```
for (inicio; final; incremento)
for (i=1; i<=10; i++)
```

Iteración

Una iteración es cada una de las repeticiones o ejecuciones individuales de un bucle. Si un bucle se ejecuta 5 veces, se dice que realizará 5 iteraciones.

Contador

Es una variable entera cuyo valor se incrementa o decrementa en una cantidad constante (generalmente 1 en 1) en cada iteración. Se utiliza para registrar la cantidad de veces que ocurre un evento o se repite un ciclo

```
contador = contador + 1
contadorPar = contadorPar + 1;
```

Acumulador

Es una variable que almacena la suma o multiplicación sucesiva de valores variables en cada iteración. Se utiliza para obtener totales, promedios o sumatorias acumulados.

```
suma = sumai;
sumo += 1
```

Control de flujo iterativo

C++ proporciona sentencias especiales para alterar la ejecución normal de una estructura repetitiva

break: Interrumpe y sale inmediatamente del bucle, ignorando las iteraciones restantes

continue: Salta el resto de las instrucciones de la iteración actual y pasa directamente a la siguiente iteración (evaluando de nuevo la condición de incremento)

Anidamiento básico de bucles

```
for (i = 1 ; i <= 10 ; i++)
  for (j=1 ; j<=10 ; j++)
    cout << i << "x" << j << "=" << j*i << endl;
```

Estructura while (mientras)

Es un bucle o ciclo de control que ejecuta un bloque de código de forma iterativa mientras se cumple una condición lógica determinada. A diferencia de otros bucles como for, el bucle while se utiliza principalmente cuando no se conoce con exactitud el número exacto de iteraciones que se van a realizar antes de iniciar la ejecución

Funcionamiento básico:

Evaluación: Antes de ejecutar el código se evalúa la condición lógica

Ejecución: Si la condición es verdadero (true), se ejecuta el cuerpo del bucle.

Repetición: Al terminar el bloque, se vuelve a evaluar la condición. El proceso continua hasta que la condición pase a ser falso (false).

Finalización: Si la condición es falsa desde el inicio, el bloque dentro del bucle no se ejecutará ni una sola vez.

Estructura repetitiva do-while (hacer-mientras)

Es un bucle o ciclo de control que ejecuta un bloque de código al menos una vez y luego continúa repitiéndola mientras se cumpla una condición lógica determinada. A diferencia del bucle while tradicional, la condición de evaluación se encuentra al final del código

Funcionamiento básica:

Ejecución obligatoria: El programa entra al ciclo y ejecuta las instrucciones del bloque directamente sin comprobar ninguna condición previa.

Evaluación posterior: Al llegar final del bosque, se evalúa la condición lógica expresada en el while

Repetición: Si la condición es verdadera (true), el flujo regresa al inicio del bloque para repetir.

Finalización: Si la condición es falsa (false), el ciclo termina y el programa continúa con la instrucción siguiente

Ejercicios

Leer n números enteros, obtener la suma total y el promedio de los números ingresados

Crear la tabla del multiplicación del 7.

Arreglos

Un arreglo (también llamado, conocido como array, vector o matriz) es una estructura de datos homogénea y de tamaño fijo que permite almacenar una colección de elementos del mismo tipo bajo un único nombre. Cada elemento individual dentro del arreglo

Arreglo unidemensional (vector)

Un arreglo unidimensionales es una colección lineal de datos que se organizan de forma secuencial uno al lado del otro en la memoria del computador. Se puede visualizar como una sola fila con múltiples casillas.

Ej: | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 2 | 14 | 15 | 16 | 20 |

→ Leer n notas, mostrar las notas ingresados y obtener promedio.

Arreglo bidimensional (matriz)

Un arreglo bidimensional es una estructura organizada en una tabla de dimensiones filas y columnas. Se puede imaginar como cuadrícula o una hoja de cálculo, donde la intersección

de una fila y una columna determina una posición o casilla específica.

Ej:

	0	1	2
0	14	15	16
1	8	13	20